Test Design (pruebas)

Conjunto de pruebas automatizadas del proyecto: comprueban cada componente por separado, sin depender del navegador ni de un servidor arrancado. Notación: Logical Design & Reverse Engineering Specification v3.

Component Design

Las pruebas viven en `src/pruebas/` y se ejecutan desde la línea de comandos (PHP y Node), sin frameworks externos. Cada fichero prueba **un** componente y usa **dobles de prueba** para no depender del mundo real: un PDO falso (no hace falta MySQL) o un `XMLHttpRequest` falso (no hace falta navegador).

```
    ------------------------------ pruebas ------------------------------
    |
    |   testLogicaDeNegocio.php   PDO falso -->  LogicaDeNegocio (business_logic)
    |   testBaseDeDatos.php       MySQL real <--  esquema de database/
    |   testAuxiliaresRest.php    --           parsearJSON() / extraerFiltros()
(communication)
    |   testEndpointsRest.php     HTTP real    endpoints de communication/
    |   pruebasLogicaFake.js      XHR falso     frontend_business_logic/ (Node)
    |
    ---------------------------------------------------------------------
```

Fichero	Componente probado	Doble de prueba	Cómo se ejecuta
`testLogicaDeNegocio.php`	`business_logic`	PDO falso	`php testLogicaDeNegocio.php`
`testBaseDeDatos.php`	`database`	— (MySQL real)	`php testBaseDeDatos.php`
`testAuxiliaresRest.php`	`communication` (auxiliares)	—	`php testAuxiliaresRest.php`
`testEndpointsRest.php`	`communication` (endpoints)	servidor HTTP real	`php testEndpointsRest.php`
`pruebasLogicaFake.js`	`frontend_business_logic`	XHR falso	`node pruebasLogicaFake.js`

Estructura de un caso de prueba

Cada fichero incluye un pequeño banco de pruebas propio (sin PHPUnit ni Jest):

```
comprobar( condicion: B, descripcion: Text ) -->
   |  suma una prueba; si condicion=false suma un fallo e imprime la descripcion
```

```
    |
    -> imprime "N pruebas, M fallos"   // M=0 => código de salida 0
```

```
testLogicaDeNegocio.php (resumen)
   |
   | 1. sustituir PDO por un PDO falso que apunta lo que se le pide
   | 2. guardarMedicion(): comprobar RN1, RN2, el INSERT parametrizado y la clave
natural
   | 3. recuperarMedicion(): comprobar filtros, orden, LIMIT y tipos de salida
   | 4. errores de BD -> resultado de dominio con clase='persistencia'
   | 5. desacoplamiento: leer el fuente y comprobar que NO hay HTTP ni JSON
```

Reglas de negocio cubiertas (trazabilidad)

Prueba	Regla / requisito	Dónde
`testLogicaDeNegocio.php`	RN1..RN5 (validación, INSERT, clave, filtros, errores)	`business_logic_design.md`
`testBaseDeDatos.php`	esquema (tabla `mediciones`, columnas, `fecha` por defecto)	`database_design.md`
`testAuxiliaresRest.php`	parseo de JSON y de filtros	`communication_design.md`
`testEndpointsRest.php`	códigos y cuerpos de la API REST	`communication_design.md`
`pruebasLogicaFake.js`	proxy/fake del frontend	`frontend_business_logic_design.md`

Design Clarifications

- Las pruebas **no** forman parte del producto: no las importa ningún componente de `src/` (salvo que se ejecuten a mano). Solo dependen de los componentes que prueban.
- Los dobles se inyectan desde fuera (un PDO falso, un `XMLHttpRequest` falso), de forma que el código de producción no sabe que está bajo prueba.
- `testLogicaDeNegocio.php` incluye además una prueba de **desacoplamiento** que lee el fuente de `business_logic` y falla si aparece cualquier rastro de transporte (`http_response_code`, `header(`, `json_encode`, `curl_`, `$_SERVER`, `$_GET`).
- Se imprime `"N pruebas, M fallos"` y el proceso devuelve código de salida `0` si `M=0` y `1` si hay algún fallo, para poder encadenarlo en un script.

General Rules

- **Programming Language:** PHP 8 (pruebas del servidor) y JavaScript ES5 (pruebas del cliente). Sin frameworks de test; banco de pruebas propio.
- **Function/Method Headers:** cada función de prueba documenta su diseño lógico en un bloque de comentario delimitado por líneas discontinuas (`--------------------`): `entrada --> funcion() --> salida`.

- **Code Readability:** código claro y autoexplicativo; cada `comprobar()` lleva una descripción en lenguaje natural de lo que se está verificando.
- **Automated Testing:** las pruebas se generan para todas las funciones críticas y se ejecutan por consola (ver `COMANDOS.txt`).